Programming with Objects & Arrays

Quick Start	Execute step
`mkdir mps`	`python mp6.py`
`cd mps`	Submit step
`mkdir 6`	`submit csc174abc 6`
`cd 6`	
`emacs mp6.py &`	
`emacs vn.py &`	

Vector Operations

The types of numbers that we've traditionally used are often referred to as *scalars*. Scalars are quite good at providing us with a sense of magnitude, but give little indication as to direction. When studying physical phenomena such as force or displacement, we need a quantitative entity that provides us with both magnitude and direction. These will be referred to as *vectors*. In all that follows, we will designate vectors with an arrow over the variable names (such as $\vec{x}$ or $\vec{y}$) and scalars using regular variables (such as x or y).

Any vector of a given magnitude and direction is equal to any other vector of the same magnitude and the same direction. It will often be convenient for us to place a vector's tail (the end without the arrow) at the origin of the Euclidean plane (i.e., $\mathbb{R}^2$) and identify the vector with the point (x, y) that is touched by its head (the end with the arrow). To avoid confusion, when we discuss a point p in the Euclidean plane, we will use parentheses as in $p = (x, y)$. On the other hand, when we are discussing a vector in $\mathbb{R}^2$ that has its head at the point (x, y) when its tail is at the origin, we will use the symbols $< x, y >$ as in $\vec{v} =< x, y >$.

Figure 1 shows $\mathbb{R}^2$ vectors $\vec{u}$ in black, $\vec{v}$ in orange and $\vec{w}$ in purple. Vectors $\vec{u}$ and $\vec{w}$ have equal directions but different magnitudes. Vectors $\vec{u}$ and $\vec{v}$ have equal magnitudes but different directions.

Vectors in $\mathbb{R}^3$

Vectors in 3 space are worthy of special attention since they play an important role in most of our physical examples. Figure 2 shows a vector $\vec{v} =< a, b, c >$ in purple. The coordinate axes in 3 space are shown in orange. A surrounding frame helps project the image of a 3 dimensional vector on the 2 dimensional screen.

Since 3 distinct points determine a unique plane, then any two non-parallel vectors will determine a unique plane in 3 space. This can be visualized by choosing the 3 points to be the two heads of the vectors and the point of intersection of their two tails. Consequently, much of our analysis of $\mathbb{R}^3$ vectors can take advantage of $\mathbb{R}^2$ techniques in the plane determined by the two $\mathbb{R}^3$ vectors. Figure 3 shows vectors $\vec{u}$ and $\vec{v}$ with their tails aligned at one point in $\mathbb{R}^3$. The orange plane is determined by that point and the two points determined by the two

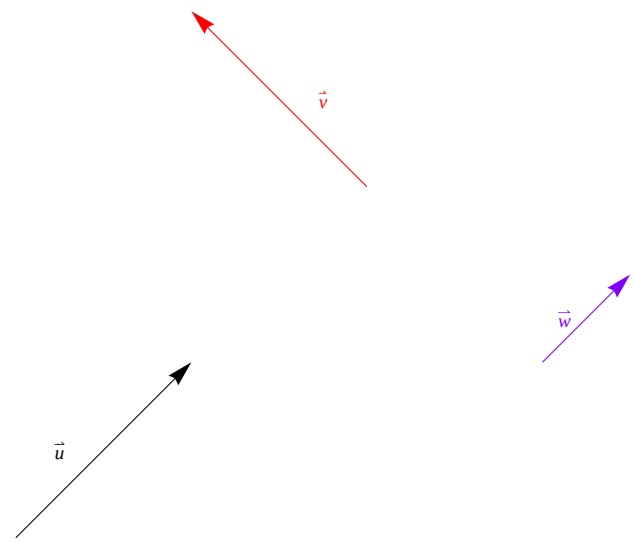

Figure 1: $\mathbb{R}^2$ Vectors with Equal Magnitude or Direction

heads of the vectors. For a more familiar example, any two of the 3 canonical unit vectors, $\vec{i} =< 1, 0, 0 >$ (in the positive x direction), $\vec{j} =< 0, 1, 0 >$ (in the positive y direction) or $\vec{k} =< 0, 0, 1 >$ (in the positive z direction) determine one of the coordinate planes in $\mathbb{R}^3$. $\vec{i}$ and $\vec{j}$ are the familiar xy plane, $\vec{i}$ and $\vec{k}$ are the xz plane and $\vec{j}$ and $\vec{k}$ are the yz plane.

Magnitude

Measurement of magnitude with real data is simply defined as the absolute value function $|x|$ or distance from the origin on the number line. With vectors, there are a number of metrics that we use to determine magnitude. Perhaps one of the most popular is the so called p-norm that is defined as follows for $v \in \mathbb{R}^3$ and positive integers p.

$$||\vec{v}||_p = (|v_1|^p + |v_2|^p + |v_3|^p)^{\frac{1}{p}} \tag{1}$$

Figure 4illustrates how "dropping" a perpendicular from the head of $\vec{v}$ to the xy-plane, shows that the 2-norm, $||\vec{v}||_2$ is the length of the hypotenuse of the right triangle determined by the points $(0, 0, 0), (a, b, 0)$ and (a, b, c). This is often referred to as the "Euclidean" norm. In addition, the line segment joining $(0, 0, 0)$ and $(a, b, 0)$ is the hypotenuse of the right triangle in the xy plane formed by the points $(0, 0, 0), (a, 0, 0)$ and $(a, b, 0)$. Since two sides of that right triangle are length a and b, then the hypotenuse has length $\sqrt{a^2 + b^2}$. Then since $||\vec{v}||_2$ is the hypotenuse of a right triangle with sides $\sqrt{a^2 + b^2}$ and c, we have Equation 2.

$$||\vec{v}||_2 = \sqrt{a^2 + b^2 + c^2} \tag{2}$$

Another popular version of the p-norm is the 1-norm which is the sum of the absolute values of the components of the vector, $\vec{v}$ as is depicted in Figure 5. This is sometimes referred to as the "taxi cab" norm and is shown in Equation 3.

$$||\vec{v}||_1 = |a| + |b| + |c| \tag{3}$$

Still another norm is known as the "infinity norm" as depicted in Figure 6 that identifies a vector's magnitude as being the maximum absolute value of the components of $\vec{v}$ as is displayed in Equation 4.

$$||\vec{v}||_\infty = \max(|a|, |b|, |c|) \tag{4}$$

Scalar multiples of vectors, $k\vec{v}$ can be thought of as stretching (for $|k| > 1$) or shrinking the vector (for $|k| < 1$)

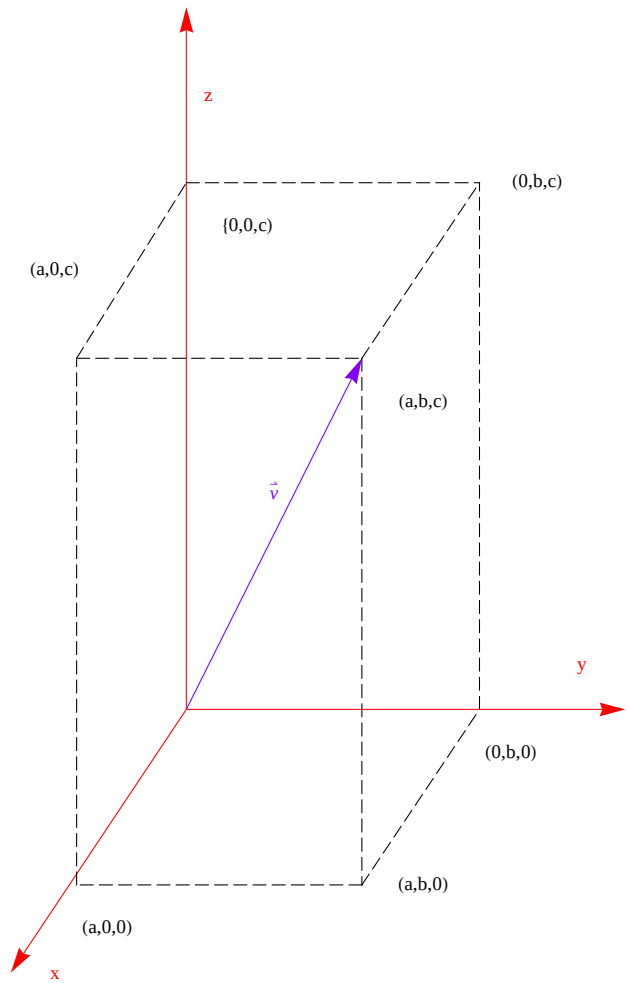

Figure 2: Vector $\vec{v} =< a, b, c >$

and possibly changing it's direction (for $k < 0$). For instance, when k is an integer, it is easy to see that $k\vec{v}$ creates k multiples of the right reference triangle bounded by $(0, 0, 0), (a, b, 0)$ and (a, b, c). In fact, this is the case for any real value of k. Consequently, we have the following relationship.

$$\begin{aligned} k\vec{v} &= k < a, b, c > \\ &= < ka, kb, kc > \end{aligned} \tag{5}$$

Figure 7 displays a *Mathematica* example that demonstrates how to create two vectors in $\vec{v}$ and $\vec{w} \in \mathbb{R}^4$ and then implements a function that returns the p-norm to display $||\vec{v}||_1$ and $||\vec{w}||_2$. After defining another function `norminf[]`, we display $||\vec{w}||_\infty$ and the scalar product $k\vec{v}$ for any $k \in \mathbb{R}$.

Zero and Unit vectors

The *zero vector* $\vec{0} =< 0, 0, 0 >$ clearly has magnitude 0. A unit vector $\vec{u}$ is any vector that has $||\vec{u}||_p = 1$. Any $\vec{v} \neq \vec{0}$ has a unit vector in the same direction as $\vec{v}$ given by $\vec{v}/||\vec{v}||_p$.

Vector arithmetic

The sum of two vectors $\vec{v}$ and $\vec{w}$ is found by placing one vector's tail at the other vector's head and forming a parallelogram with two copies of each vector. The diagonal that begins at the corner with two tails of the vectors

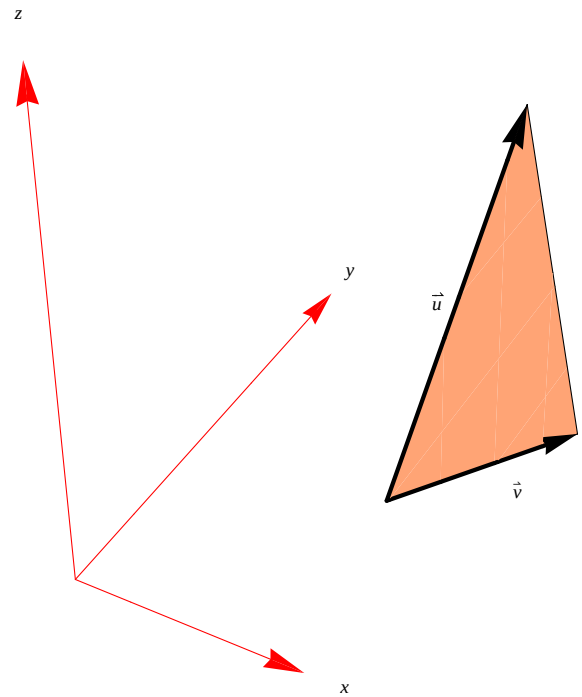

Figure 3: The Plane Determined by Vectors $\vec{u}$ and $\vec{v}$

and ends at the corner with two heads of the vectors is the vector sum. It can be shown that for $\mathbb{R}^3$ vectors $\vec{v} =< v_1, v_2, v_3 >$ and $\vec{w} =< w_1, w_2, w_3 >$, the vector sum satisfies the following formula.

$$\vec{v} + \vec{w} =< v_1 + w_1, v_2 + w_2, v_3 + w_3 > \tag{6}$$

Figure 8 shows the parallelogram method of determining the sum of vector $\vec{v}$ in black and vector $\vec{w}$ in orange as elements of $\mathbb{R}^2$. The vector sum $\vec{v} + \vec{w}$ is the diagonal that travels from the corner with the tails of $\vec{v}$ and $\vec{w}$ to the corner with the heads and is shown in purple. The right triangles with side v_1 and v_2 and sides w_1 and w_2 attempt to geometrically justify the vector sum $\vec{v} + \vec{w}$ in the first two dimensions.

Figure 9 shows the sum $\vec{v} + \vec{w}$ in $\mathbb{R}^3$ with supporting triangles of height v_3 and w_3 that geometrically justify the third dimension.

Consequently, vector subtraction can be thought of as the following.

$$\begin{aligned} \vec{v} - \vec{w} &= \vec{v} + (-1)\vec{w} \\ &= < v_1, v_2, v_3 > +(-1) < w_1, w_2, w_3 > \\ &= < v_1, v_2, v_3 > + < -w_1, -w_2, -w_3 > \\ &= < v_1 - w_1, v_2 - w_2, v_3 - w_3 > \end{aligned} \tag{7}$$

Vector Dot Product

One can motivate the creation of the vector dot product by asking the following question. "When a force that is not horizontal is exerted on a horizontal plane, what is the resultant horizontal force?" For instance, when we push an object on a horizontal table with a force that is not horizontal, how much of the force is parallel to the table? If the direction of the table surface is given by the vector $\vec{w}$ and the direction of the force is given by the vector $\vec{v}$, then we are asking for the vector that is a multiple of $\vec{w}$ that is generated by a perpendicular that is "dropped" from the head of $\vec{v}$ to $\vec{w}$. This is often referred to as the vector projection of $\vec{v}$ onto $\vec{w}$ which we will denote as $\overrightarrow{Proj}_{\vec{v}}\vec{w}$. All of this analysis can take place in the plane determined by $\vec{v}$ and $\vec{w}$, but we will discuss

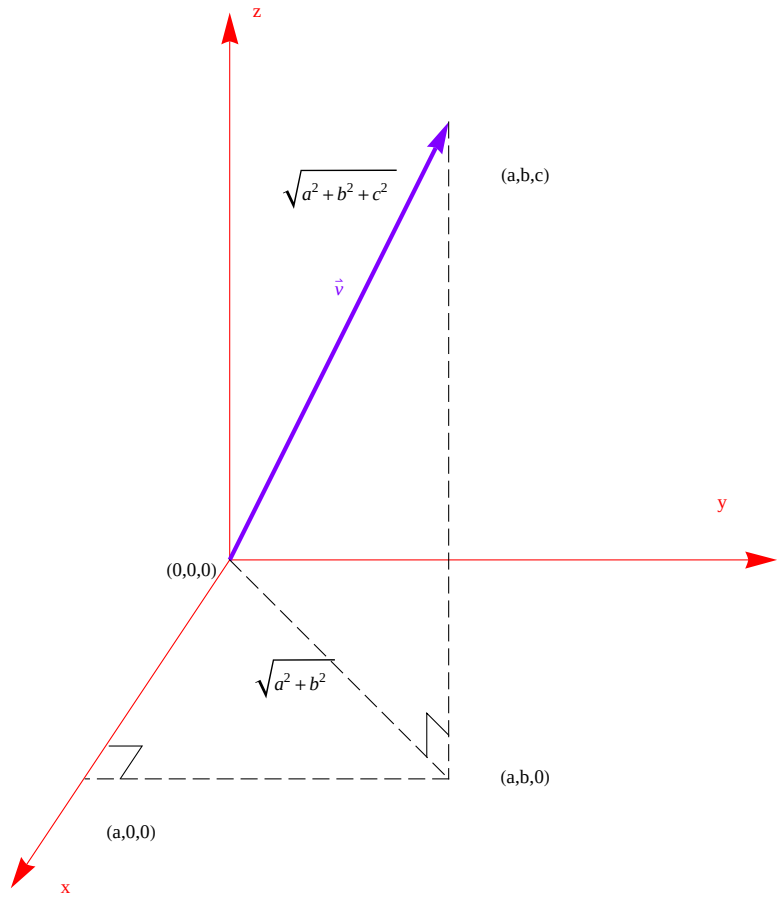

Figure 4: The Euclidean Norm $|| < a, b, c > ||_2 = \sqrt{a^2 + b^2 + c^2}$

these vectors as elements of $\mathbb{R}^3$. Since $\overrightarrow{Proj}_{\vec{v}}\vec{w}$ is in the same direction as vector $\vec{w}$, it must be equal to $\alpha\vec{w}$ for some real value of α. Then, assuming that $\vec{v} =< v_1, v_2, v_3 >$ and $\vec{w} =< w_1, w_2, w_3 >$, we have a right triangle that is bounded by vectors $\vec{v}$, $\alpha\vec{w}$ and $\alpha\vec{w} - \vec{v}$ that due to Pythagoras, and Equation 2, satisfies the following relationship.

$$\begin{aligned} ||\vec{v}||_2^2 &= ||\alpha\vec{w}||_2^2 + ||\alpha\vec{w} - \vec{v}||_2^2 \\ &= \alpha^2 w_1^2 + \alpha^2 w_2^2 + \alpha^2 w_3^2 + (\alpha w_1 - v_1)^2 + (\alpha w_2 - v_2)^2 + (\alpha w_3 - v_3)^2 \end{aligned}$$

and so we have

$$\begin{aligned} v_1^2 + v_2^2 + v_3^2 &= \alpha^2 w_1^2 + \alpha^2 w_2^2 + \alpha^2 w_3^2 + \alpha^2 w_1^2 - 2\alpha v_1 w_1 + v_1^2 + \alpha^2 w_2^2 - 2\alpha v_2 w_2 + v_2^2 + \alpha^2 w_3^2 - 2\alpha v_3 w_3 + v_3^2 \\ &= v_1^2 + v_2^2 + v_3^2 + 2\alpha^2(w_1^2 + w_2^2 + w_3^2) - 2\alpha(v_1 w_1 + v_2 w_2 + v_3 w_3) \end{aligned}$$

and so either $\alpha = 0$ or $\alpha(w_1^2 + w_2^2 + w_3^2) - v_1 w_1 - v_2 w_2 - v_3 w_3 = 0$, the last of which gives Equation 8.

$$\alpha = \frac{v_1 w_1 + v_2 w_2 + v_3 w_3}{w_1^2 + w_2^2 + w_3^2} \tag{8}$$

We'll define the *dot product* of two vectors $\vec{a} =< a_1, a_2, a_3 >$ and $\vec{b} =< b_1, b_2, b_3 >$ as the following scalar value.

$$\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + a_3 b_3 \tag{9}$$

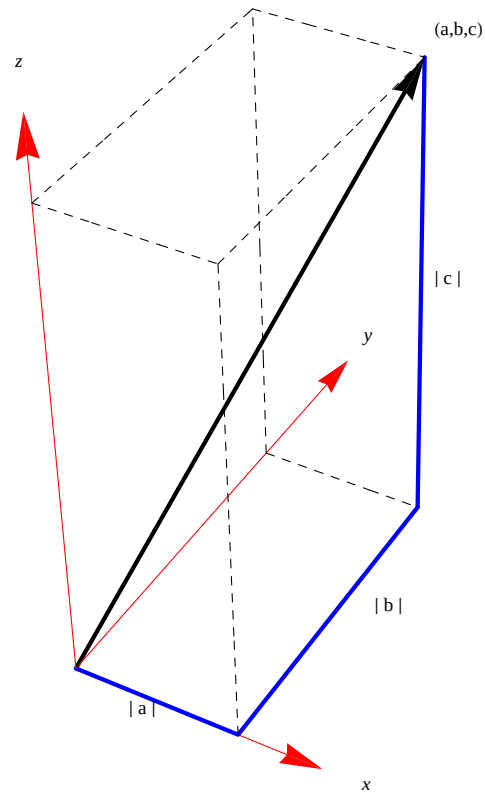

Figure 5: The Taxi Cab Norm $|| < a, b, c > ||_1 = |a| + |b| + |c|$

Then, Equation 8 becomes the following.

$$\alpha = \frac{\vec{v} \cdot \vec{w}}{\vec{w} \cdot \vec{w}} \tag{10}$$

The dot product inherits commutativity and associativity from the multiplication operation on the reals. The dot product satisfies a distributive property such that, if $\vec{u} =< u_1, u_2, u_3 >$, $\vec{v} =< v_1, v_2, v_3 >$ and $\vec{w} =< w_1, w_2, w_3 >$ with $\alpha, \beta \in \mathbb{R}$ we have the following.

$$\begin{aligned}
\vec{u} \cdot (\alpha\vec{v} + \beta\vec{w}) &= < u_1, u_2, u_3 > \cdot (\alpha < v_1, v_2, v_3 > + \beta < w_1, w_2, w_3 >) \\
&= < u_1, u_2, u_3 > \cdot (< \alpha v_1, \alpha v_2, \alpha v_3 > + < \beta w_1, \beta w_2, \beta w_3 >) \\
&= < u_1, u_2, u_3 > \cdot < \alpha v_1 + \beta w_1, \alpha v_2 + \beta w_2, \alpha v_3 + \beta w_3 > \\
&= \alpha u_1 v_1 + \beta u_1 w_1 + \alpha u_2 v_2 + \beta u_2 w_2 + \alpha u_3 v_3 + \beta u_3 w_3 \\
&= (\alpha u_1 v_1 + \alpha u_2 v_2 + \alpha u_3 v_3) + (\beta u_1 w_1 + \beta u_2 w_2 + \beta u_3 w_3) \\
&= \alpha(u_1 v_1 + u_2 v_2 + u_3 v_3) + \beta(u_1 w_1 + u_2 w_2 + u_3 w_3) \\
&= \alpha(< u_1, u_2, u_3 > \cdot < v_1, v_2, v_3 >) + \beta(< u_1, u_2, u_3 > \cdot < w_1, w_2, w_3 >) \\
&= \alpha(\vec{u} \cdot \vec{v}) + \beta(\vec{u} \cdot \vec{w})
\end{aligned} \tag{11}$$

Last of all, we note the following.

$$\begin{aligned}
\vec{v} \cdot \vec{v} &= < v_1, v_2, v_3 > \cdot < v_1, v_2, v_3 > \\
&= v_1^2 + v_2^2 + v_3^2 \\
&= \left(\sqrt{v_1^2 + v_2^2 + v_3^2}\right)^2
\end{aligned}$$

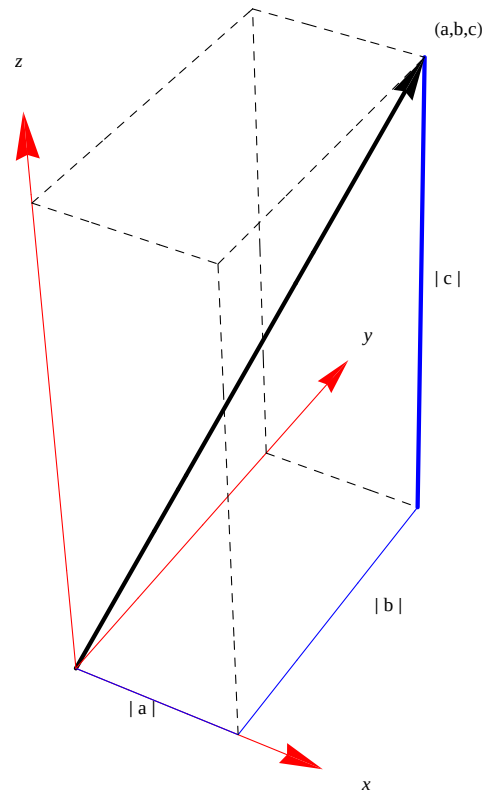

Figure 6: The Infinity Norm $|| < a, b, c > ||_\infty = \max(|a|, |b|, |c|)$

$$= \quad ||\vec{v}||_2^2 \tag{12}$$

Figure 10 shows the vector projection of vector $\vec{v}$ in black onto vector $\vec{w}$ in orange. The vector projection $\overrightarrow{\text{Proj}}_{\vec{w}}\vec{v}$ is shown in purple.

Angle θ between two vectors

Let θ be the counter-clockwise angle measured from vector $\vec{v}$ to vector $\vec{w}$. Then we have a right triangle with sides $||\vec{v}||_2$, $||\vec{w}||_2$ and $||\vec{v} - \vec{w}||_2$. The law of cosines states the following.

$$||\vec{v} - \vec{w}||_2^2 = ||\vec{v}||_2^2 + ||\vec{w}||_2^2 - 2||\vec{v}||_2||\vec{w}||_2 \cos\theta \tag{13}$$

Now, by Equation 11 and Equation 12, we have the following.

$$\begin{aligned} ||\vec{v} - \vec{w}||_2^2 &= (\vec{v} - \vec{w}) \cdot (\vec{v} - \vec{w}) \\ &= \vec{v} \cdot (\vec{v} - \vec{w}) - \vec{w} \cdot (\vec{v} - \vec{w}) \\ &= \vec{v} \cdot \vec{v} - \vec{v} \cdot \vec{w} - \vec{w} \cdot \vec{v} + \vec{w} \cdot \vec{w} \\ &= ||\vec{v}||_2^2 - 2\vec{v} \cdot \vec{w} + ||\vec{w}||_2^2 \end{aligned} \tag{14}$$

Equating Equation 13 and Equation 14 gives the following.

$$||\vec{v}||_2^2 - 2\vec{v} \cdot \vec{w} + ||\vec{w}||_2^2 \quad = \quad ||\vec{v}||_2^2 + ||\vec{w}||_2^2 - 2||\vec{v}||_2||\vec{w}||_2 \cos\theta$$

This then leads to

$$-2\vec{v} \cdot \vec{w} \quad = \quad -2||\vec{v}||_2||\vec{w}||_2 \cos\theta$$

```
(* Vectors v & w *)
v = {1, 2, -1, -2}
{1, 2, -1, -2}

w = {2, -4, 0, -3}
{2, -4, 0, -3}

(* Function normp[] and demonstrations *)
normp[x_, p_] := Sum[Abs[x[[i]]]^p, {i, 1, Length[x]}]^(1.0/p)

normp[v, 1]
6.

normp[w, 2]
5.38516

norminf[x_] := Max[Abs[x]]
norminf[w]
4

(* Scalar k times vector v *)
MatrixForm[k * v]
```

$$\begin{pmatrix} k \\ 2k \\ -k \\ -2k \end{pmatrix}$$

Figure 7: *Mathematica* Vectors and Norms

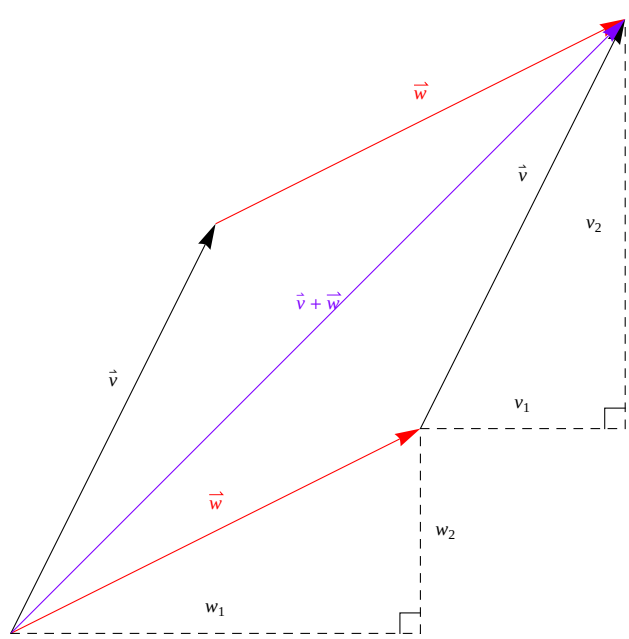

Figure 8: The Vector Sum $\vec{v} + \vec{w}$ in $\mathbb{R}^2$

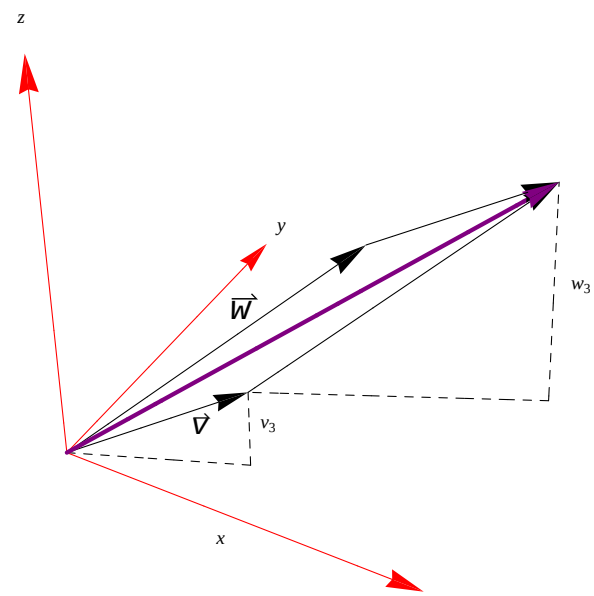

Figure 9: The Vector Sum $\vec{v} + \vec{w}$ in $\mathbb{R}^3$

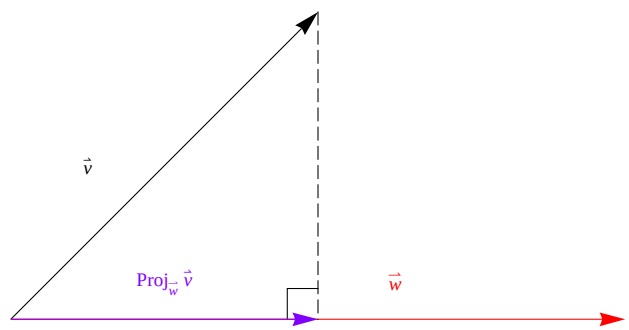

Figure 10: Projecting Vector $\vec{v}$ onto Vector $\vec{w}$

and solving for $\cos\theta$ gives the following.

$$\cos\theta = \frac{\vec{v}\cdot\vec{w}}{||\vec{v}||_2||\vec{w}||_2} \tag{15}$$

Equation 15 indicates that vectors $\vec{v}$ and $\vec{w}$ are perpendicular ($\theta = \pi/2$), if and only if $\vec{v}\cdot\vec{w} = 0$. In higher dimensional spaces, we say that the vectors $\vec{v}$ and $\vec{w}$ are *orthogonal*. Figure 11 displays two $\mathbb{R}^3$ vectors $\vec{v}$ and $\vec{w}$ and the angle θ determined by them.

The tensor product of vectors $\vec{v} =< v_1, v_2, v_3 >$ and $\vec{u} =< u_1, u_2, u_3 >$ is given by the following.

$$\begin{aligned}\vec{v}\otimes\vec{u} &= (v_i u_j)_{\substack{1\le i\le 3\\ 1\le j\le 3}} \\ &= \begin{pmatrix} v_1u_1 & v_1u_2 & v_1u_3 \\ v_2u_1 & v_2u_2 & v_2u_3 \\ v_3u_1 & v_3u_2 & v_3u_3 \end{pmatrix}\end{aligned} \tag{16}$$

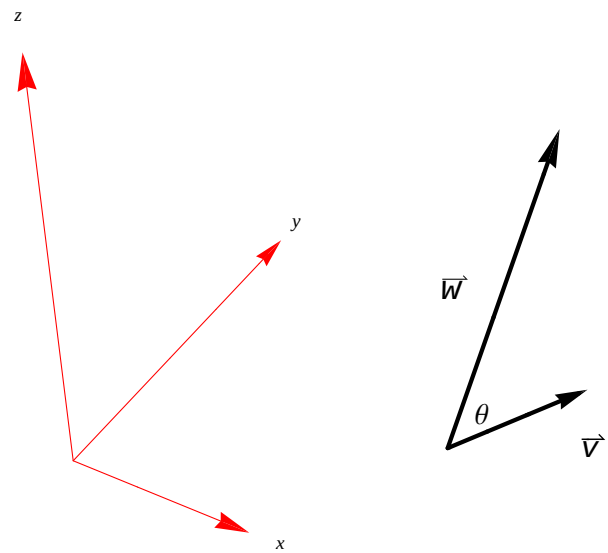

Figure 11: $\mathbb{R}^3$ Vectors $\vec{v}$ and $\vec{w}$ and the Angle θ Determined by Them

Figure 12 displays a *Mathematica* example that reproduces the initialization of the two $\mathbb{R}^4$ vectors $\vec{v}$ and $\vec{w}$ and then displays the vector sum $\vec{v}+\vec{w}$, the vector difference $\vec{v}-\vec{w}$, the vector dot product $\vec{v}\cdot\vec{w}$, and the tensor product $\vec{v}\otimes\vec{w}$

Vectors in $\mathbb{R}^n$

Although the geometry is difficult to visualize, most of the operations that we've discussed extend quite easily to vectors in an n-space. So, we'll let our vectors $\vec{v}$ and $\vec{w}$ be defined as follows.

$$\begin{aligned}\vec{v} &= < v_1, v_2, \ldots, v_n > \\ \vec{w} &= < w_1, w_2, \ldots, w_n >\end{aligned}$$

Then, Equation 2 through Equation 7 as well as Equation 9 are extended to the following.

$$\begin{aligned}\vec{v}+\vec{w} &= < v_1+w_1, v_2+w_2, \ldots, v_n+w_n > \\ k\vec{v} &= < kv_1, kv_2, \ldots, kv_n > \\ \vec{v}-\vec{w} &= < v_1-w_1, v_2-w_2, \ldots, v_n-w_n > \\ \vec{v}\cdot\vec{w} &= v_1w_1+v_2w_2+\cdots+v_nw_n \\ \vec{v}\otimes\vec{w} &= \left(v_iw_j\right)_{\substack{1\le i\le n\\ 1\le j\le n}} \\ &= \begin{pmatrix} v_1w_1 & v_1w_2 & \ldots & v_1w_n \\ v_2w_1 & v_2w_2 & \ldots & v_2w_n \\ \vdots & \vdots & \ddots & \vdots \\ v_nw_1 & v_nw_2 & \ldots & v_nw_n \end{pmatrix}\end{aligned}$$

Equation 11, Equation 12 and Equation 15 extend to higher dimensional spaces with no changes.

```
(* Vectors v & w *)
v = {1, 2, -1, -2}
{1, 2, -1, -2}

w = {2, -4, 0, -3}
{2, -4, 0, -3}

(* Vector sum *)
MatrixForm[v + w]
```

$$\begin{pmatrix} 3 \\ -2 \\ -1 \\ -5 \end{pmatrix}$$

```
(* Vector difference *)
MatrixForm[x - v]
```

$$\begin{pmatrix} -1 \\ 6 \\ -1 \\ 1 \end{pmatrix}$$

```
(* Vector dot product *)
v.w
0

(* Tensor product of v and w *)
MatrixForm[Outer[Times, v, w]]
```

$$\begin{pmatrix} 2 & -4 & 0 & -3 \\ 4 & -8 & 0 & -6 \\ -2 & 4 & 0 & 3 \\ -4 & 8 & 0 & 6 \end{pmatrix}$$

Figure 12: *Mathematica* Vector Operations and Tensor Product

Problem Statement and Specifications

When working with lists of data, programming languages must have access to large numbers of variables without the burden of large numbers of variable names. In a mathematics course, we'll use indexed variables for these applications. In the Python programming language, we use the `list` data structure to support our needs for processing arrays, vectors and matrices. When we work with scalar data (`int`, `float`, `string` or `boolean`), programming languages have a number of operators and functions built in to them so that we can perform popular arithmetic operations. When we increase the complexity of the data structure to address vector or matrix operations, we find some support but could always use more.

Along with the `list` data structure, we'd like to introduce the notion of object-oriented programming techniques within the Python language. At this time, we're in a position to discuss *encapsulation* that describes how the data that we choose to manipulate is closely associated with the functions that manipulate the data. This is the nature of an object supporting the storage and access of both data attributes and functions. In addition, we will occasionally need to control the access to the object using private attributes or we can provide maximal access and use public attributes.

One of the reasons that object-oriented programming has become popular within the industry has to do with the manner in which our software ages. A valuable object-oriented design will provide support for us to maintain our code so that it can be modified easily as it ages, thus avoiding the introduction of new errors as we change the code to support new usage. Quite often, we introduce other object-oriented strategies to achieve these goals. *Inheritance* is a mechanism that object-oriented languages support so that new objects can be derived from existing objects. *Polymorphism* represents the ability for values of different data types to be handled using a

uniform interface. Our goal will be to provide whatever flexibility that we can by means of encapsulation, but we will not be able to address inheritance or polymorphism in this exercise.

Assignment 6 Statement: Use the `vn` object defined in the file `vn.py` and modify the program in `mp6.py` that inputs an $\mathbb{R}^4$ vector and calculates and displays the p-norm, the ∞-norm, the dot product, the cosine of the hyper-angle determined by the vectors and the scalar product of an $\mathbb{R}^4$ vector. In addition your program should calculate and display the sum, difference and the tensor product of two $\mathbb{R}^4$ vectors. Specifically, you will need to correctly manipulate the `vn` object to code the vector sum, scalar product, the cosine of the angle determined by the vectors, the infinity norm and the dot product.

Explanations of the vector formulae are given in the mathematical notes. There is no other input other than one $\mathbb{R}^4$ vector. For any two vectors $\vec{v}$ and $\vec{w}$, the output will consist of vector quantities $\vec{v}$, $\vec{w}$, $\vec{v}+\vec{w}$, $\vec{v}-\vec{w}$, $k\cdot\vec{v}$ for $k\in\mathbb{R}$, and scalar values $\cos(\theta)$, $||\vec{v}||_1$, $||\vec{w}||_2$, $||\vec{w}||_\infty$, $\vec{v}\cdot\vec{w}$ as well as the value of $\vec{v}\otimes\vec{w}$

Assignment 6 Specifications:

Create an initialized 4 dimensional vector $\vec{v}$

Input **A 4 dimensional vector $\vec{w}$**

Assign $\vec{x}=\vec{v}-\vec{w}$

Output $\vec{v}$ $\quad\vec{w}$ $\quad\vec{v}+\vec{w}$ $\quad\vec{v}-\vec{w}$ $\quad 2.0\cdot\vec{v}$

$\cos(\theta)$ $\quad||\vec{v}||_1$ $\quad||\vec{w}||_2$ $\quad||\vec{w}||_\infty$ $\quad\vec{v}\cdot\vec{w}$

$\vec{v}\otimes\vec{w}$

The Object `vn`

Before we discuss the various programs and subprograms that need to manipulate the `vn` objects, we should discuss the construction of the object itself. This occurs in the file `vn.py` in a Python `class`. The `vn` object will use a `list` data attribute capable of storing 4 `float` values called `vecn`. Since our goal is to design as flexible a class as possible, we'll use another attribute as an `int` constant called `VECLENGTH` that is set to the value of `4` and make every attempt to design the class and supporting functions in terms of `VECLENGTH`. This way, if we wish to use the `vn` class for the creation of vectors of a different length, we'll change the value of `VECLENGTH`.

The creation of the `vn` class requires the use of a `class` statement in the file `vn.py` that looks like the following.

```
class class_name:

    statement_1
    statement_2
    ⋮
    statement_n
```

The value of `VECLENGTH` should be available to everyone to control our loops so we'll want that to be declared as `public`. However the stored vector should not allow direct access, so we'll need to declare that as `private`. In particular, our `class`, so far, looks like the following.

```
class vn:

    VECLENGTH = 4
```

Constructors/Initializers

The support of object declaration and initialization is the responsibility of the `__init__` function. Our `__init__` function initializes instances of a `vn` object with VECLENGTH `float` items in a `list`. The name of the function is the same for all objects created in Python. The function supports `VECLENGTH` parameters, each of which is assigned to the underlying `list` `vecn` and uses a default value of `0.0` for each item. Note `__init__` function has no return type. The following lines of code invokes the `__init__` function. The variable `x` is declared as a `vn` object and explicitly identities the `4` parameters, where the variable `y` will be declared with the default initialization.

```
x = vn(1.0, 2.0, -1.0, -2.0)
w = vn()
```

Destructors

The role of releasing the memory associated with an object when execution leaves an object's scope is the responsibility of the `__del__` function. In the case of the `vn` object, we simply allow the garbage collection mechanism to invoke the `__del__` function when all references to the object are deleted.

```
class vn:
    # vector length (to be modified as needed)
    VECLENGTH = 4

    # The __init__ function initializes the attributes
    # receives: 4 foat values set to 0.0 by default
    #           if called with 0 parameters
    # returns:  a new vn object with the 4 values of vecn initialized
    def __init__(self, a = 0.0, b = 0.0, c = 0.0, d = 0.0):
        # private list data member of size VECLENGTH
        self.__vecn = [0.0] * self.VECLENGTH
        self.__vecn[0] = a
        self.__vecn[1] = b
        self.__vecn[2] = c
        self.__vecn[3] = d
```

Extractors/Accessor

The purpose of the extractor is to return information about the object without modifying the object. This is how we'll support knowledge of what data is stored in the `vn` without providing direct access to the array `vecn`. We can name the extractor any valid Python name, but it is very popular to name an extractor `get()`. Given a declared `vn` object named `v` and an `int` value named `n`, `get()` is implemented as a member function that has `n` as a parameter. `get()` is invoked by using the following within an expression.

```
v.get(n)
```

The code to implement this method is placed into an area that is equivalent to where we place `vn` constructors.

```
# get() extractor to return a vn element
# receives: int n to index into the vn
# returns: n'th vn element
def get(self, n):
    if 0 <= n and n < self.VECLENGTH:
        # return the n'th element of vecn
        return self.__vecn[n]
    else:
        print("get method received bad argument : ", n)
        return -1.0
```

Mutators

The purpose of the mutator is to modify information stored in the object. This is how we'll support modifying the data stored in the `vn` without providing direct access to the array `vecn`. We can name the mutator any valid Python name, but it is very popular to name a mutator `set()`. Given a declared `vn` object named `v`, an `int` value named `n` and a `float` value named `y`, `set()` is implemented as a member method that has `n` and `y` as parameters. `set()` is invoked by executing the following statement.

```
v.set(n, y)
```

The code to implement this function is placed into an area that is equivalent to where we placed method `get()`.

```
# set() mutator to change a vn element
# receives: int n to index into vn and store double y
# returns: void, but updates instantiation parameter
def set(self, n, y):
    # store the n'th element of vecn
    if 0 <= n and n < self.VECLENGTH:
        self.__vecn[n] = y
    else:
        print("set method received bad argument : ", n)
```

Assignments

The vast majority of class implementations can fulfill all the needs of software users who use them with constructors, destructors, extractors and mutators. Once objects are instantiated with the constructor, we provide whatever access the user needs to manipulate the object with the extractors and mutators. When the object needs to be returned to the free memory pool, the destructor becomes active. However, it's not unusual to provide a few more features to our objects and we'll do the same with the `vn` object. The first task we'll take on is the assignment of one `vn` object to another. Essentially, we'll need to accomplish this by copying each of the `vecn` vector elements from one `vn` object to the next. To accomplish this, so we will create another member function of the `vn` class. We'll follow the tradition of other assignment operations and have the data move from the right hand object to the left hand object. In this case, the data is being copied from the parenthesized parameter, which is considered the right hand object to the instantiation parameter, which is considered the left hand object. For declared `vn` objects `w` and `x` with `w` having been initialized, the invocation of the operation is coded as follows.

```
x.equals(w)
```

The implementation of the assignment within the `vn` class follows.

```
# equals() function to copy all vecn elements from vn vsource
# receives: vn vsource
# returns: void, but copies vsource attribute vecn to the vecn
#          attribute of the vn object that calls the equals function
def equals(self, vsource):
    # visit each element of vsource and copy to vecn
    for i in range(self.VECLENGTH):
        self.__vecn[i] = vsource.get(i)
```

Input and Output

The last tasks that we'll take on will be input and output of `vn` objects. For input we'll need to read across a horizontal line to pick up the various elements of the object. The parameter list is void and our member function simply loops over the `VECLENGTH` of the `vn` object and inputs each `float` into the `vecn` `list`. The input operation is invoked as follows for a `vn` object `w`.

```
w.input()
```

The implementation of the operation is as follows.

```
# input() function to read a vn from the keyboard
# receives: void
# process: prompts and reads 4 float values from keyboard
#          console input; updates vn attribute vecn
# returns: void
def input(self):
    print("Enter (one at a time) ", self.VECLENGTH, " reals:", sep="")
    for i in range(self.VECLENGTH):
        self.__vecn[i] = float(input())
```

For output, the `Python` language takes advantage of the fact that all displayed objects have a `string` representation. The strategy is to use a member function called `__str__` that returns a `string` representation of the object to be displayed. Consequently, the function will not need any parameters. The invocation of the output function will look as follows with an initialized `vn` object `v`.

```
print(v)
```

The code to implement the output operation looks as follows.

```
# __str__ function to convert a vn object to a string
# receives: void
# process: accesses the vecn attribute
# returns: formatted string appropriate for output
def __str__(self):
    return f"{self.__vecn[0]:6.1f}{self.__vecn[1]:6.1f}{self.__vecn[2]:6.1f}{self.__vecn[3]:6.1f}"
```

Object Analysis and Algorithmic Development

Since we've defined the object `vn`, we'll move on now to discuss the `main` function. Essentially, our goal is to test out every component of the class, either directly or indirectly, as well as add some other types of functionality. Our pseudocode for the `main` function is as follows.

1. *output* **a prompt for and** *input* $\vec{w}$

2. *output* $\vec{w}$ **& initialized** $\vec{v}$

3. *output* $||\vec{v}||_1, ||\vec{w}||_2, ||\vec{w}||_\infty, \vec{v} \cdot \vec{w}$

4. *output* $\cos(\theta)$ **determined by** $\vec{v}$ **&** $\vec{w}$

5. *output* $k \cdot \vec{v}$ **for** $k = 2$

6. *output* $\vec{v} + \vec{w}$

7. *process* $\vec{x} = \vec{v} - \vec{w}$

8. *output* $\vec{x}$

9. *process* $\vec{v} \otimes \vec{w}$

10. *output* $\vec{v} \otimes \vec{w}$

All of the steps for the vector difference, the p-norm and the tensor product have been coded as functions. For this assignment, you are to code the vector sum, the scalar product the dot product and the cosine of the hyper-angle determined by the vectors $\vec{v}$ and $\vec{w}$. These steps are identified in red in the following algorithm of the `main` function.

Pseudocode for the `main` function

Statement	Variables involved
1) prompt for & input $\vec{w}$	`string` constant & `vn` $\vec{w}$
2) display $\vec{w}$ & initialized $\vec{v}$	`vn` objects $\vec{v}$ & $\vec{w}$
3) calculate & display `double` scalars $||\vec{v}||_1$, $||\vec{w}||_2$,$||\vec{w}||_\infty$ & $\vec{v} \cdot \vec{w}$	`vn` objects $\vec{v}$ & $\vec{w}$
4) calculate & display $\cos(\theta)$ determined by $\vec{v}$ & $\vec{w}$	`vn` objects $\vec{v}$ & $\vec{w}$
5) calculate & display $k \cdot \vec{v}$ for $k = 2$	`float` k and `vn` $\vec{v}$
6) calculate & display $\vec{v} + \vec{w}$	`vn` objects $\vec{v}$ & $\vec{w}$
7) process $\vec{x} = \vec{v} - \vec{w}$	`vn` objects $\vec{x}, \vec{v}$ and $\vec{w}$
8) display $\vec{x}$	`vn` object $\vec{x}$
9) calculate $\vec{v} \otimes \vec{w}$	`vn` objects $\vec{v}$ & $\vec{w}$ and a 4 by 4 `float` array
10) display $\vec{v} \otimes \vec{w}$	a 4 by 4 `float` array

For each of the operations that must be coded, we'll interact with the `vn` objects. Some of our requirements such as the scalar product or the vector sum, require a `vn` answer. Others, such as the infinity norm, cosine of a hyper-angle or the dot product, will return a scalar. All of these types of calculations can be generalized as list operations. We consider our `vn` objects as lists that are of equal length. To formulate our results, we use a repetition statement to travel the length of the `list` and compute our answer as we visit each element of the . Usually, the Python repetition statement involved is nothing more than a `for` statement. As we visit each `list` element, we'll interact with the various `vn` methods such as `get()` or `set()`. We'll begin with the **List Revision Algorithm** which will return a list in the form of a `vn` object.

List Revision Algorithm
Returns a list based upon computations using one or more other lists

Loop over the indices of the list $0 \leq i <$ `VECLENGTH`

Invoke `get()` to retrieve the i'th element of the `vn` arguments
Process the retrieved elements to revise the list appropriately
Invoke `set()` to store the result in the i'th position of the answer list

End the repetition structure

As an example, let's consider the vector difference that evaluates $\vec{a}-\vec{b}$ for two initialized `vn` objects $\vec{a}$ and $\vec{b}$. This will obviously have 2 parameters of type `vn` and we'll arbitrarily assume that the left hand parameter should be $\vec{a}$ and the right hand parameter should be $\vec{b}$. We'll need a loop that retrieves the `float` elements of $\vec{a}$ and $\vec{b}$ starting at position 0 and continuing on to position `VECLENGTH-1`. Once we've retrieved the i'th elements of vectors $\vec{a}$ and $\vec{b}$, we subtract them and store the result into the i'th position of a temporary `vn` object. After we've traveled through the entire list, we simply return the temporary `vn`. The `diffvn()` function to evaluate this is shown below.

```
def diffvn(a, b):
    temp = vn()
    # visit each of a and b and store the difference a minus b
    for i in range(vn.VECLENGTH):
        temp.set(i, a.get(i) - b.get(i))
    return temp
```

This is the algorithm that we can use any time that the answer is a `vn` or `list`. This includes the scalar product and the vector sum. It will also come into play multiple times when we construct the tensor product. For those algorithms that result in a scalar value, we'll resort to the **List Reduction Algorithms**. There are usually 4 that can calculate the sum, the product, the maximum value and the minimum value of a `list`.

List Reduction Algorithm
Returns a scalar based upon computations using one or more other lists

Initialize the answer to an identity value

Loop over the indices of the list $0 \leq i <$ `VECLENGTH`

Invoke `get()` to retrieve the i'th element of the `vn` arguments
Update the answer to incorporate the value retrieved

End the repetition structure

Let's use the p-norm function for an example of this algorithm. There will be two parameters, one of type `vn`, call it $\vec{a}$ and the other an `int`, call it p. Since this is a sum, we initialize the temporary return value to the additive identity, 0.0. Within the loop that generates indices over $0 \leq i <$ `VECLENGTH`, after we extract the i'th element from our `vn` parameter, we increment the temporary by the p'th power of the absolute value of the element. After leaving the loop, we return the p'th root of the temporary variable. The `normp()` method is shown below.

```
def normp(a, p):
    temp = 0.0
    if p > 0:
        # after initializing reduction variable, update by visiting
        # each element in the vn argument raised to the p'th power
        for i in range(vn.VECLENGTH):
            temp += math.pow(math.fabs(a.get(i)), p)
        # return the p'th root of the sum
        return math.pow(temp, 1.0/p)
    else:
        print("normp() recieved p = ", p, ", returning 0", sep="")
        return temp
```

Coding for Compilation

Logon to your `csc.oakton.edu` account and create a new subdirectory of the `mps` directory called `6` to work on your assignment.

```
cd mps
mkdir 6
cd 6
```

The code in the file `vn.py` defines the `vn` object to be manipulated. The code in the file `mp6.py` accesses the `vn` object by means of executing the `mp6.py` file in the same directory that contains the file `vn.py` that defined the `vn` object. Note the use of the import statement in the `mp6.py` file which imports the `vn` class for use of the `vn` object.

```
from vn import vn
```

Copy these files over to your directory and start an editor session with the following commands.

```
cp /samples/csc/174/mps/6/* .
emacs mp6.py &
```

After updating the documentation to reflect your name, execute the program to examine its behavior. Click on your `xterm` window to access your command line prompt and execute your program with the command shown below.

```
python mp6.py
```

Follow the Pseudocode - Coding the `main` function

As we follow the pseudocode for the `main` function, we'll need to implement certain functions. In those cases, the statement in the `main` function has been commented out and you simply need to remove the comment after the implementing the function. In other cases, the statement in the `main` function is missing and we'll need to code that.

Step 1

`# Test the input function`

This consists of using the input function to prompt the user for input and respond appropriately. This part of the program is already coded for you.

```
w.input()
```

Step 2

```
# Test the output method
```

This consists of using the `__str__` function to display the value of a `vn` object on the monitor. This part of the program is already coded for you.

```
print(w)
```

Step 3

```
# Invocation of scalar functions:  norms, dot, etc.
```

```
print(f"v norm 1 = {normp(v, 1):4.1f}\nw norm 2 = {normp(w, 2):4.1f}")
```

As the value returned by each function invocation is a `float` scalar, we can format that value as any other `float` scalar. The actual code in the support files contains `string` constants embedded within the `print` statements that identifies the values being displayed.

```
print(f"w norm inf = {norminf(w):4.1f}\nv dot w = {dot(v, w):4.1f")
```

The code necessary to implement the value of $\vec{v} \cdot \vec{w}$ should use the List Reduction Algorithm that was mentioned above. It is very similar to the p-norm algorithm where we initialize the return value to 0 and as we loop over the indices of the parameters, we update by the expression $v_i w_i$. Code the `dot` function in the area indicated by the comments in the file `mp6.py`.

Similarly, the code necessary to implement the value of $||\vec{w}||_\infty$ should again use the List Reduction Algorithm, but this time initialize the return value to $|w_0|$. As you loop over the indices of $\vec{w}$, compare the return value to $|w_i|$ and update if you find a value larger than the current return value. Code the ∞-norm function in the area indicated by the comments in the file `mp6.py`.

Checkpoint

To ensure that the methods that return scalars are working correctly, you should execute your program with the following commands.

```
python mp6.py
```

If you've correctly coded the dot product and the ∞ norm, then you can move on to Step 4. Otherwise, you'll need to debug those functions before continuing. If the program compiles but returns incorrect values of these functions, the areas to examine would include that the loops run over the entire length of the indices (i.e. $0 \leq i \leq n-1$) and that the return values are correctly initialized (0 for the dot product, $|w_0|$ for the ∞ norm).

Step 4

```
# Code the display of cos(theta)
```

At this point, the methods for the dot product and the 2-norm have been implemented and all that is necessary is to code an output statement that evaluates and displays the cosine of the hyper-angle that is determined by vectors $\vec{v}$ and $\vec{w}$. Code this output statement before continuing.

$$\cos\theta = \frac{\vec{v}\cdot\vec{w}}{||\vec{v}||_2||\vec{w}||_2}$$

Step 5

```
# Invoke and display scalar product
```

Since the output of `vn` objects is available to us, we can invoke the functions into output statements to test their execution. These lines are commented out in the file `mp6.py` and can simply be uncommented to invoke the functions.

```
print("2 * v =", scalarvn(2, v))
```

To implement the scalar multiple function, you'll want to use the List Revision Algorithm discussed above. Within the function, it might be easiest to generate a temporary `vn` object and loop over the indices of the parameter setting the i'th element of the temporary `vn` to the product of the scalar times the i'th element of the `vn` parameter. Code the function that will implement the scalar product in the area of `mp6.py` that the comments indicate before continuing.

To implement the vector sum of two `vn` parameters in the function, you once again store the result in a temporary `vn` object setting the i'th element of the temporary `vn` equal to the sum of the i'th elements of the parameter `vn` objects. Code the function that will implement the sum of two `vn` objects in the area of `mp6.py` that the comments indicate before continuing.

Step 6

```
# Invoke and display vector sum
```

Once the vector sum of two `vn` objects has been implemented as a function, it can be invoked within an output statement as other functions have that return a `vn` object. This output statement is commented out for you in the file `mp6.py`. Uncomment this output statement before continuing.

```
print("v + w =", sumvn(v, w))
```

Step 7

```
# Test assignment and vector difference
```

Both the vector difference of two `vn` objects and the assignment of one `vn` object to another were coded for you in the files `mp6.py` and `vn.py`. At this step, we simply assign the difference between $\vec{v}$ and $\vec{w}$ to $\vec{x}$.

```
x.equals(diffvn(v,  w))
```

Step 8

```
# Display vector difference
```

This step simply displays the `vn` object $\vec{x}$ that was set to the value of $\vec{v}-\vec{w}$ in Step 7 above.

```
print("v - w =", x)
```

Step 9

```
# Invoke tensor product
```

The invocation of function `tensor` simply sends the `vn` objects $\vec{v}$ and $\vec{w}$ as parameters. The result is returned and assigned to `tensorvw` the two dimensional `list` of type `float`.

```
tensorvw = tensor(v, w)
```

The implementation of function `tensor` begins with the generation of the two dimensional `list` that will hold the return value. After that, we simply enter a nesting of `for` loops and extract the appropriate elements of the parameters to multiply together for storage in the `tens` array. Note that due to the row dominant storage nature of Python two dimensional lists, we ensure that the left most index is counted in the outermost loop.

```
def tensor(a, b):
    # assign a float 4x4 array
    tens = [[0.0]*vn.VECLENGTH]*vn.VECLENGTH
    # visit each array element and initialize with ai * bj
    for i in range(vn.VECLENGTH):
        for j in range(vn.VECLENGTH):
            tens[i][j] = a.get(i)*b.get(j)
    return tens
```

Step 10

```
# Display tensor product
```

The display of the two dimensional `list` `tensorvw` is simply a nesting of `for` loops that again ensures the left most index in the `list` is counted in the outermost loop.

```
print("v tensor w =")
for i in range(vn.VECLENGTH):
    # loop over each column
    for j in range(vn.VECLENGTH):
        # loop over each row
        print(f"{tensorvw[i][j]:6.1f}", end="")
    print()
```

Printing and submitting

Once you are satisfied with the correctness of your program, print it as you did with previous assignments by using the following command that assumes that you are working in the room 1234 at Oakton. Retrieve your copy from the printer.

```
printer  1234  mp6.py
```

Finally, submit your program with the following command that assumes that you are registered in section *abc* of CSC 174.

```
submit  csc174abc  6
```

Copyright © 1999-2015 by Robert Walter Sompolski